RTSP Stream

RAPPORT TECHNIQUE

SYSTÈME MULTI-RTSP DYNAMIQUE

Serveur C++ (GStreamer) + Interface Python (PyQt)

Rechargement dynamique

Réalisé par	**Ayoub Smayen**
Fonction	Développeur C++
Spécialisation	Streaming vidéo, architectures embarquées
Technologies	GStreamer, C++17, Python, PyQt, OpenCV
Version	2.0
Date	Avril 2026

Document confidentiel - Tous droits réservés

Table des matières

1 Introduction **5**
1.1 Contexte et objectifs 5
1.2 Architecture globale 5

2 Étude d'existant **6**
2.1 Analyse comparative 6
2.2 Conclusion 6

3 Diagramme de Cas d'Utilisation **7**
3.1 Diagramme UML 7
3.2 Description des cas d'utilisation 7

4 Diagramme de Séquence **8**
4.1 Rechargement automatique de la configuration 8
4.2 Rechargement côté visualisation PyQt 8

5 Diagramme de Classes **9**
5.1 Classes du serveur C++ 9
5.2 Classes de l'interface visualisation (PyQt) 10

6 Architecture du Système **11**
6.1 Vue d'ensemble 11
6.2 Pipeline GStreamer 11

7 Installation et Déploiement **12**
7.1 Prérequis 12
7.2 Compilation 12
7.2.1 Linux / macOS 12
7.2.2 Windows 12

8 Plan de Tests **14**

9 Conclusion **15**
9.1 Bilan 15
9.2 Points forts 15
9.3 Apport personnel 15
9.4 Évolutions prévues 16

Annexes **17**
Annexe A : Extrait de code – Serveur C++ 17
Annexe B : Extrait de code – Visualisation PyQt 17
Annexe C : Fichier de configuration JSON 18

Table des figures

3.1 Diagramme de cas d'utilisation 7

4.1 Diagramme de séquence - Rechargement serveur 8
4.2 Diagramme de séquence - Rechargement visualisation 8

5.1 Diagramme de classes - Serveur C++ 9
5.2 Diagramme de classes - Visualisation PyQt 10

6.1 Architecture générale du système 11
6.2 Pipeline GStreamer par canal 11

Liste des tableaux

2.1 Comparaison des solutions existantes 6

3.1 Cas d'utilisation - Administrateur 7

7.1 Prérequis par plateforme 12

8.1 Plan de tests fonctionnels 14

1 Introduction

1.1 Contexte et objectifs

Le protocole **RTSP (Real Time Streaming Protocol)** est un standard de l'industrie pour la diffusion de médias en temps réel. Ce projet consiste à développer une solution complète de diffusion et visualisation multi-flux RTSP.

Objectifs du projet

- Diffusion de flux vidéo multi-canal sur réseau local ou Internet
- Compatibilité avec les lecteurs standards (VLC, FFplay, OBS)
- Configuration dynamique **sans interruption du service**
- Interface d'administration intuitive
- Incrustation de texte personnalisée par canal

1.2 Architecture globale

Le système se décompose en deux parties indissociables :

1. **Serveur RTSP multi-canal** (C++17 + GStreamer)
   - Diffusion de flux vidéo (tests, fichiers, caméras)
   - Rechargement à chaud de la configuration
   - Incrustation texte et horodatage par canal
2. **Interface de visualisation dynamique** (Python / PyQt + OpenCV)
   - Chargement automatique des caméras depuis JSON
   - Mise à jour des flux sans redémarrage
   - Thread par caméra pour garantir la fluidité

2 Étude d'existant

2.1 Analyse comparative

Table 2.1 – Comparaison des solutions existantes

Solution	RTSP	Multi-flux	UI	Reconfigurable	Customisable
VLC Media Player					
OpenCV seul					
ZoneMinder					
FFmpeg + SDL					
OBS Studio					
Notre solution					

2.2 Conclusion

Aucune solution open-source simple ne combine :

- Multi-RTSP temps réel
- Interface utilisateur moderne
- Rechargement dynamique sans coupure
- Extensibilité (C++ pour le cœur, Python pour l'UI)

D'où la pertinence de notre architecture hybride.

3 Diagramme de Cas d'Utilisation

3.1 Diagramme UML

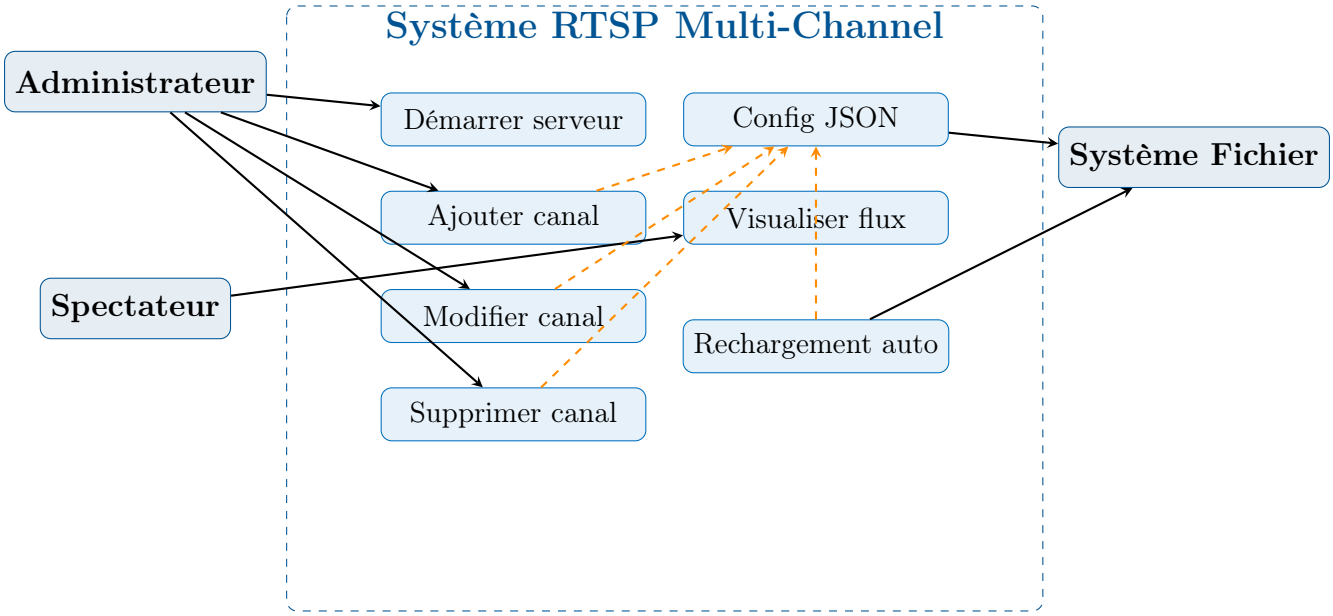

FIGURE 3.1 – Diagramme de cas d'utilisation

3.2 Description des cas d'utilisation

TABLE 3.1 – Cas d'utilisation - Administrateur

UC-01 : Démarrer le serveur RTSP	
Acteur principal	Administrateur
Pré-condition	Exécutable compilé, fichier config valide
Post-condition	Serveur actif, canaux disponibles en RTSP
Scénario nominal	1. L'admin lance la GUI 2. Il clique sur "Démarrer serveur" 3. Le serveur C++ se lance 4. Les flux deviennent accessibles

4 Diagramme de Séquence

4.1 Rechargement automatique de la configuration

Figure 4.1 – Diagramme de séquence - Rechargement serveur

4.2 Rechargement côté visualisation PyQt

Figure 4.2 – Diagramme de séquence - Rechargement visualisation

5 Diagramme de Classes

5.1 Classes du serveur C++

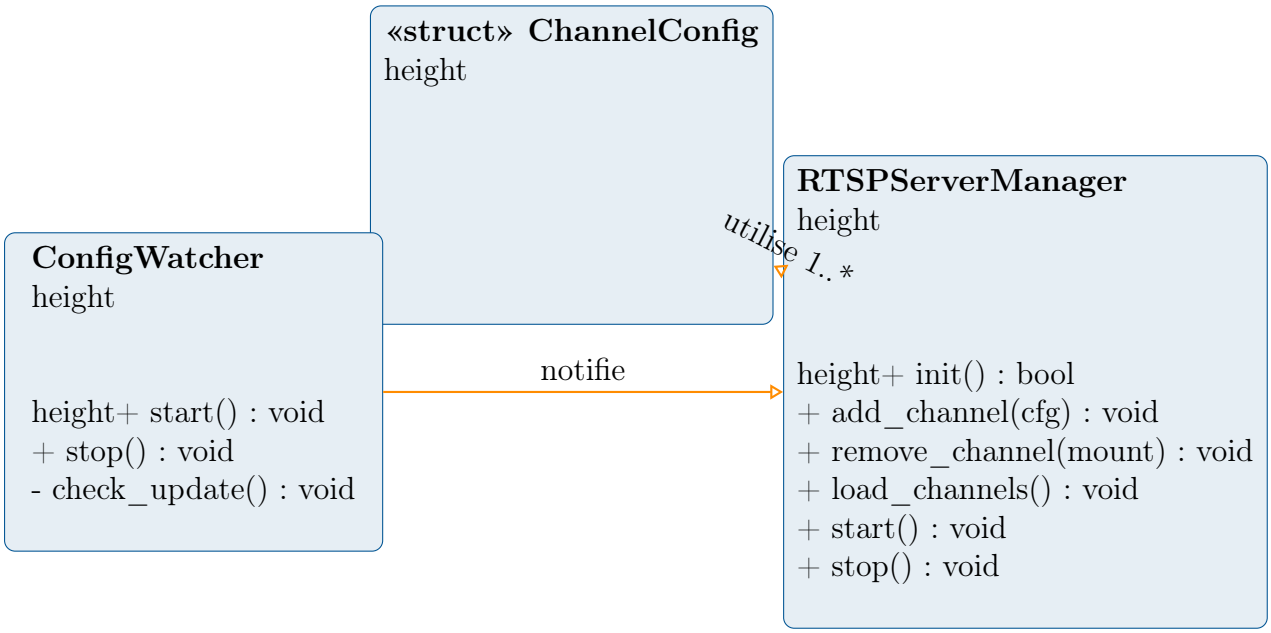

FIGURE 5.1 – Diagramme de classes - Serveur C++

5.2 Classes de l'interface visualisation (PyQt)

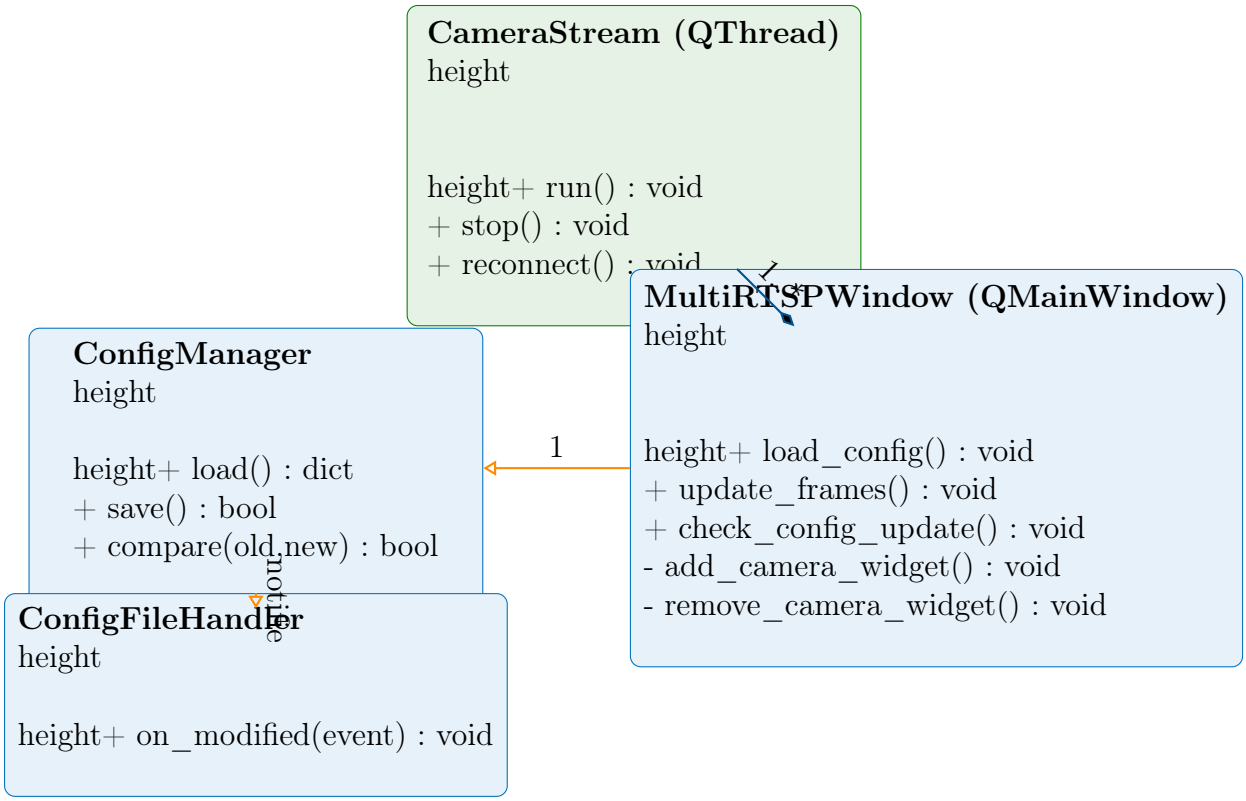

FIGURE 5.2 – Diagramme de classes - Visualisation PyQt

6 Architecture du Système

6.1 Vue d'ensemble

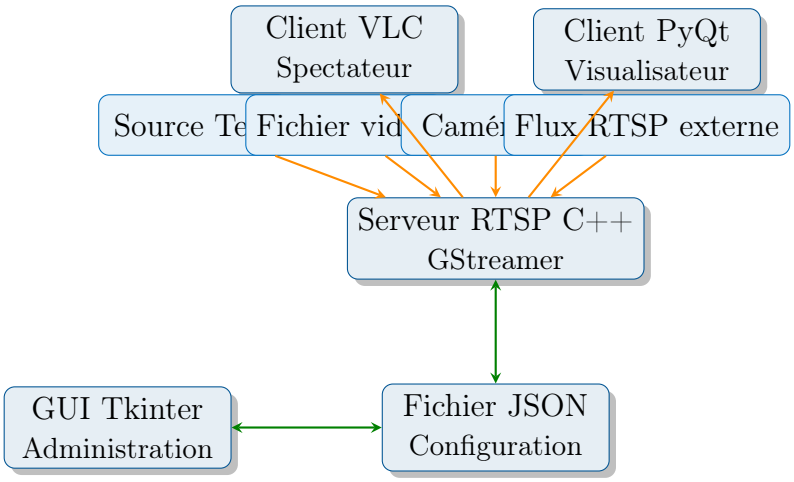

FIGURE 6.1 – Architecture générale du système

6.2 Pipeline GStreamer

FIGURE 6.2 – Pipeline GStreamer par canal

7 Installation et Déploiement

7.1 Prérequis

Table 7.1 – Prérequis par plateforme

Composant	Windows	Linux	macOS
GStreamer	`gstreamer-1.0.msi`	`apt install`	`brew install`
CMake	`cmake-3.16+.msi`	`apt install`	`brew install`
Python	`python-3.10+.exe`	`apt install`	`brew install`
OpenCV	`pip install`	`pip install`	`pip install`
PyQt5	`pip install`	`pip install`	`pip install`
watchdog	`pip install`	`pip install`	`pip install`

7.2 Compilation

7.2.1 Linux / macOS

Listing 7.1 – Compilation sous Linux/macOS

```
# Installation des d pendances
sudo apt install libgstreamer1.0-dev libgstreamer-rtsp-server-1.0-
    dev
sudo apt install cmake build-essential python3 python3-pip

# Compilation
cmake -B build
cmake --build build

# Lancement
./build/rtsp_server config/rtsp_config.json
```

7.2.2 Windows

Listing 7.2 – Compilation sous Windows

```
cmake -B build -DGST_ROOT="C:/Program Files/gstreamer/1.0/
    msvc_x86_64"
cmake --build build --config Release
```

```

# Lancement
.\build\Release\rtsp_server.exe config\rtsp_config.json
```

8 Plan de Tests

Table 8.1 – Plan de tests fonctionnels

ID	Cas de test	Résultat attendu	Statut
T-01	Démarrer serveur sans config	Message d'erreur	**OK**
T-02	3 canaux simultanés	URLs accessibles	**OK**
T-03	Modification du port	Redémarrage sur nouveau port	**OK**
T-04	Ajout canal via GUI	Visible en < 3s	**OK**
T-05	Désactivation canal	Non accessible	**OK**
T-06	Modification overlay texte	Changement instantané	**OK**
T-07	10 flux simultanés	Stable < 200ms	**OK**
T-08	Ouverture VLC depuis GUI	VLC lance le flux	**OK**
T-09	Copie URL	URL dans presse-papier	**OK**
T-10	Arrêt propre	Clients déconnectés	**OK**
T-11	Rechargement config	Changements < 3s	**OK**
T-12	Chargement cameras.json	Flux affichés	**OK**
T-13	PyQt - Ajout caméra	Nouveau flux apparaît	**OK**
T-14	Reconnexion auto	Reconnexion après perte	**OK**

9 Conclusion

9.1 Bilan

Ce projet apporte une **réponse complète et innovante** au besoin de diffusion et de visualisation multi-flux RTSP avec reconfiguration dynamique.

9.2 Points forts

- **Séparation des responsabilités** : cœur performant en C++, interface flexible en Python
- **Rechargement à chaud** des deux côtés (serveur et visualisation)
- **Compatibilité standards** : VLC, FFplay, OBS
- **Extensibilité** : facile d'ajouter des fonctionnalités
- **Multiplateforme** : Windows, Linux, macOS

9.3 Apport personnel

Ayoub Smayen – Développeur C++

En tant que développeur C++, j'ai conçu et implémenté :

- Le serveur RTSP basé sur `gst-rtsp-server`
- Le watcher de fichier config avec rechargement sans interruption
- Le pipeline GStreamer optimisé pour la faible latence
- L'interface Python/PyQt pour la visualisation dynamique
- L'interface Tkinter pour l'administration

9.4 Évolutions prévues

Fonctionnalité	Statut
Support caméras IP natives (rtspsrc)	**Prévu**
Authentification RTSP	**Prévu**
Enregistrement des flux	**Prévu**
Interface web (Flask + WebSockets)	**Prévu**
Détection de mouvement (OpenCV)	**Prévu**
Notification en cas de perte de flux	**Prévu**

Annexes

Annexe A : Extrait de code – Serveur C++

Listing 9.1 – RTSPServerManager.cpp

```
void RTSPServerManager::add_channel(const ChannelConfig& cfg) {
    if (!cfg.enabled) return;

    GstRTSPMediaFactory* factory = gst_rtsp_media_factory_new();

    std::string pipeline = cfg.source;
    if (cfg.source == "videotestsrc") {
        pipeline += " pattern=" + std::to_string(cfg.pattern);
    }
    pipeline += " ! video/x-raw ! clockoverlay ! textoverlay text=\"" +
                cfg.text_overlay + "\" ! videoconvert ! x264enc ! rtph264pay name=pay0 pt=96";

    gst_rtsp_media_factory_set_launch(factory, pipeline.c_str());
    gst_rtsp_media_factory_set_shared(factory, TRUE);

    gst_rtsp_mount_points_add_factory(mounts_, cfg.mount.c_str(), factory);
    g_object_unref(factory);

    g_print("    Canal ajoute : %s -> %s\n", cfg.name.c_str(), cfg.mount.c_str());
}
```

Annexe B : Extrait de code – Visualisation PyQt

Listing 9.2 – camera$_s$*tream.py*

```
import cv2
from PyQt5.QtCore import QThread, pyqtSignal, QMutex

class CameraStream(QThread):
    frame_updated = pyqtSignal(object)

```

```
    def __init__(self, url: str, parent=None):
        super().__init__(parent)
        self.url = url
        self.running = True
        self.mutex = QMutex()

    def run(self):
        cap = cv2.VideoCapture(self.url)
        while self.running:
            ret, frame = cap.read()
            if ret:
                self.frame_updated.emit(frame)
            else:
                cap.release()
                self.msleep(1000)
                cap = cv2.VideoCapture(self.url)
        cap.release()

    def stop(self):
        self.mutex.lock()
        self.running = False
        self.mutex.unlock()
```

Annexe C : Fichier de configuration JSON

Listing 9.3 – rtsp$_{c}onfig.json$

```
{
  "server": {
    "host": "0.0.0.0",
    "port": 8554
  },
  "channels": [
    {
      "id": 1,
      "name": "Camera_Principale",
      "mount": "/main",
      "source": "videotestsrc",
      "pattern": 18,
      "text_overlay": "Live - Camera Principale",
      "enabled": true
    },
    {
      "id": 2,
      "name": "Camera_Secondaire",
      "mount": "/secondary",
      "source": "filesrc location=/home/video/test.mp4 ! decodebin
        ",
      "pattern": 0,
      "text_overlay": "Replay - Camera Secondaire",
```

```
      "enabled": true
    }
  ]
}
```